Verilang - Gramatica Corregida (version final)

Nicolas Daza (202421626) | Gabriel Puyo (202420863)

ISIS-2111 PLE | Proyecto 2 - Sintaxis Concreta en Rascal

1. Cambios respecto a la versión anterior (Proyecto 1)

La implementacion en Rascal introdujo ajustes respecto a la gramatica entregada en el Proyecto 1. A continuacion se describe cada cambio.

Cambio A - Expresion unifica todo: Cuantificador, Aplicacion, ExpresionBinaria y Operador se eliminan como no-terminales independientes

En la gramatica del Proyecto 1, Cuantificador, Aplicacion, Argumento, ExpresionBinaria y Operador eran no-terminales separados. En la implementacion Rascal, todos se fusionan dentro de un unico no-terminal Expresion usando bloques de prioridad separados por el simbolo mayor-que. Esto elimina la ambiguedad de forma declarativa

```
-- Antes:
Expresion ::= Cuantificador | ExpresionBinaria | Aplicacion | '(' Expresion ')'
ExpresionBinaria ::= Expresion operador Expresion
Cuantificador ::= ('forall'|'exists') Id ('in' Id)? '.' Expresion
Aplicacion ::= Id Argumento+
Argumento ::= Id | '(' Aplicacion ')'
operador ::= '*' | '/' | '+' | '-' | '**' | ... | 'and' | 'or' | '->' | ...
```

```
-- Despues (todo en Expresion con prioridades):
syntax Expresion
= ident: Id name
| paren: '(' Expresion e ')'
> left app: Expresion lhs Expresion rhs
> right exp: Expresion lhs '**' Expresion rhs
> left mul/div/mod ...
> left add/sub ...
> non-assoc comparaciones ...
> left andOp / orOp ...
> right impl / implFat ...
> right forallIn | forallSimple | existsIn | existsSimple;
```

Cambio B - DeclaracionOperador acepta atributos como variante opcional

El Proyecto 1 elimino los atributos de los operadores. Sin embargo, el ejemplo del enunciado en el Proyecto 1 muestra: **'defoperator union : Set -> Set -> Set [associative id:0] end'**, lo que requiere aceptarlos. La solucion final ofrece dos alternativas: con atributos y sin atributos.

```
-- Antes:
```

```
DeclaracionOperador ::= 'defoperator' Id ':' FirmaOperador 'end'
```

```
-- Despues:
DeclaracionOperador ::= 'defoperator' Id ':' FirmaOperador Atributos 'end'
| 'defoperator' Id ':' FirmaOperador 'end'
```

Cambio C - DeclaracionRegla usa Expresion (no Aplicacion)

El Proyecto 1 propuso usar el no-terminal “Aplicación” en las reglas. En la implementacion, Aplicacion ya no existe como no-terminal independiente (fue absorbido por Expresion). Por tanto, DeclaracionRegla usa Expresion en ambos lados, lo que cubre los mismos casos.

```
-- Antes:
DeclaracionRegla ::= 'defrule' '(' Aplicacion ')' '->' '(' Aplicacion ')' 'end'
```

```
-- Despues:
DeclaracionRegla ::= 'defrule' '(' Expresion ')' '->' '(' Expresion ')' 'end'
```

Cambio D - DeclaracionExpresion tiene dos variantes explicitas (con y sin atributos)

La version anterior usaba Atributos? como sufijo opcional. En Rascal las dos formas se declaran como alternativas con etiquetas distintas para facilitar el mapeo al AST.

```
-- Antes:
DeclaracionExpresion ::= 'defexpression' Expresion (Atributos)? 'end'
```

```
-- Despues:
DeclaracionExpresion ::= 'defexpression' Expresion Atributos 'end'
| 'defexpression' Expresion 'end'
```

Cambio E - Atributo tiene una tercera variante con valor vacio (simbolo de conjunto vacio)

El ejemplo del enunciado del Proyecto 1 contiene el elemento vacío, es decir, un atributo cuyo valor es el simbolo de conjunto vacio. Se añade una tercera alternativa para cubrir este caso.

```
-- Antes:
Atributo ::= Id | Id ':' Id
```

```
-- Despues:
Atributo ::= Id | Id ':' Id | Id ':' empty-set-symbol
```

Cambio F - 'neg' eliminado de las palabras reservadas

'neg' aparecia como palabra reservada en el Proyecto 1 pero en la implementacion final se omitio del bloque keyword Reserved, ya que actua como operador dentro de Expresion y no requiere proteccion como palabra clave.

```
-- Antes:
keyword Reserved = ... | 'and' | 'or' | 'neg';
```

```
-- Despues:
keyword Reserved = ... | 'and' | 'or';
```

2. Conjuntos formales

Terminales

```
Terminales = {
"defmodule", "using", "defspace", "defoperator",
"defvar", "defrule", "defexpression", "defrelation",
"end", "forall", "exists", "in",
"*", "/", "+", "-", "**", "%",
"<", ">", "<=", ">=", "<>", "=", "equiv",
"and", "or", "->", "=>", ".",
"(", ")", "[", "]", ":", ",", "empty-set",
identificador, letra, digito
}
```

No-Terminales

```
No-Terminales = {
Programa, Modulo, Importacion, Declaracion,
DeclaracionEspacio, DeclaracionOperador, DeclaracionRelacion,
DeclaracionAtributos, FirmaOperador,
Atributos, Atributo,
DeclaracionVariables, Variable,
DeclaracionRegla, DeclaracionExpresion,
Expresion
}
```

```
Simbolo de inicio: Programa
```

NOTA: Cuantificador, Aplicacion, Argumento, ExpresionBinaria y Operador fueron eliminados como no-terminales independientes. Sus construcciones estan integradas directamente en Expresion con prioridades (ver Cambio A).

3. Reglas de producción

- Programa y Modulo

```
Programa ::= Modulo
```

```
Modulo ::= 'defmodule' identificador
(Importacion)*
(Declaracion)*
'end'
```

```
Importacion ::= 'using' identificador
```

- Declaracion

```
Declaracion ::= DeclaracionEspacio
| DeclaracionOperador
| DeclaracionVariables
| DeclaracionRegla
| DeclaracionExpresion
| DeclaracionRelacion
| DeclaracionAtributos
```

- Espacios

```
DeclaracionEspacio ::= 'defspace' identificador 'end'
| 'defspace' identificador '<' identificador 'end'
```

- Operadores - con atributos opcionales (Cambio B)

```
DeclaracionOperador ::= 'defoperator' identificador ':' FirmaOperador Atributos 'end'
| 'defoperator' identificador ':' FirmaOperador 'end'
FirmaOperador ::= identificador ('->' identificador)+
```

- Relaciones

```
DeclaracionRelacion ::= 'defrelation' identificador ':' FirmaOperador 'end'
```

- Atributos - con variante vacia (Cambio E)

```
DeclaracionAtributos ::= Atributos
Atributos ::= '[' Atributo+ ']'
Atributo ::= identificador
| identificador ':' identificador
| identificador ':' empty-set-symbol
```

- Variables

```
DeclaracionVariables ::= 'defvar' Variable (',' Variable)* 'end'
Variable ::= identificador ':' identificador
```

- Reglas - sobre Expresion (Cambio C)

```
DeclaracionRegla ::= 'defrule' '(' Expresion ')' '->' '(' Expresion ')' 'end'
```

- Expresiones - con atributos opcionales (Cambio D)

```
DeclaracionExpresion ::= 'defexpression' Expresion Atributos 'end'
| 'defexpression' Expresion 'end'
```

- Expresion - unificada con prioridades (Cambio A)

```
Expresion ::=
```

```
identificador -- atomo
| '(' Expresion ')'
```

```
> Expresion Expresion
```

```
> Expresion '**' Expresion
```

```
> Expresion '*' Expresion
```

```
| Expresion '/' Expresion
| Expresion '%' Expresion
```

```
> Expresion '+' Expresion
```

```
| Expresion '-' Expresion
```

```
> Expresion '<' Expresion
| Expresion '>' Expresion
| Expresion '<=' Expresion
| Expresion '>=' Expresion
| Expresion '=' Expresion
| Expresion '<>' Expresion
| Expresion 'equiv' Expresion
| Expresion 'in' Expresion
```

```
> Expresion 'and' Expresion
```

```
> Expresion 'or' Expresion
```

```
> Expresion '->' Expresion
```

```
| Expresion '=>' Expresion
```

```
> 'forall' identificador 'in' identificador '.' Expresion
| 'forall' identificador '.' Expresion
| 'exists' identificador 'in' identificador '.' Expresion
| 'exists' identificador '.' Expresion
```

- *Tokens*

```
identificador ::= letra (letra | digito | '-')*
```

```
letra ::= 'a'..'z' | 'A'..'Z'
digito ::= '0'..'9'
```